

Exhibit*Guide*

From lo-fi wireframes to a working Django application — where the project stands today, two live demos of what it can do, and the road to a React + Django REST version for final submission.

Presentation 4 · ≤ 15 minutes

2 live code demos

Django 4.2 · Python

TODAY, IN FIFTEEN MINUTES

<i>i.</i>	The journey — then, now, next	2 min
<i>ii.</i>	Before / after & what's built	3 min
<i>iii.</i>	Register a new user, live	DEMO 3 min
<i>iv.</i>	Under the hood — URLs, views, models	2 min
<i>v.</i>	Run the test suite, live	DEMO 2 min
<i>vi.</i>	Feedback acted on · the final stretch	3 min

Five checkpoints: idea, pivot, first code, *working app*, decoupled app.

Each presentation moved the project a stage further. Today is the last checkpoint before final submission — a working server-rendered Django app, about to be split into a **React front end talking to a Django REST API**.

P1 · BRIEF



Concept & wireframes

Museum audio-guide idea; 10-frame scan-to-content wireframe flow; MVP scoped around CRUD.

P2 · PIVOT



Commercial galleries

Repositioned as visitor intelligence & lead capture for independent galleries — save becomes a warm lead.

P3 · UPDATE 1



First Django build

Project scaffold, first models, Django admin as the gallery CMS, scan → exhibit page rendering.

TODAY · P4

Working monolith

Full auth flow, collector dashboard, watchlist & collections, inquiry → prospect leads, automated tests.

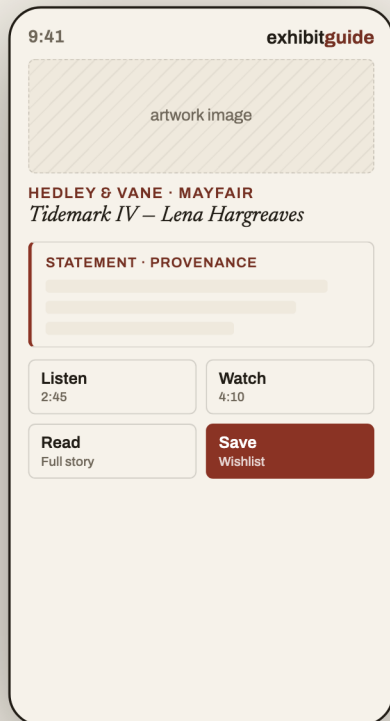
FINAL SUBMISSION

React + Django REST

Same models, decoupled: DRF API + JWT behind a React PWA front end. The final stretch.

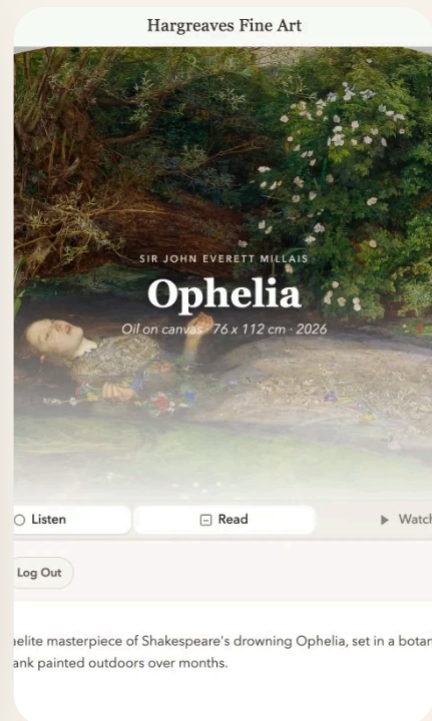
The P1 sketch is now a page *served by Django*.

Then P1 lo-fi wireframe



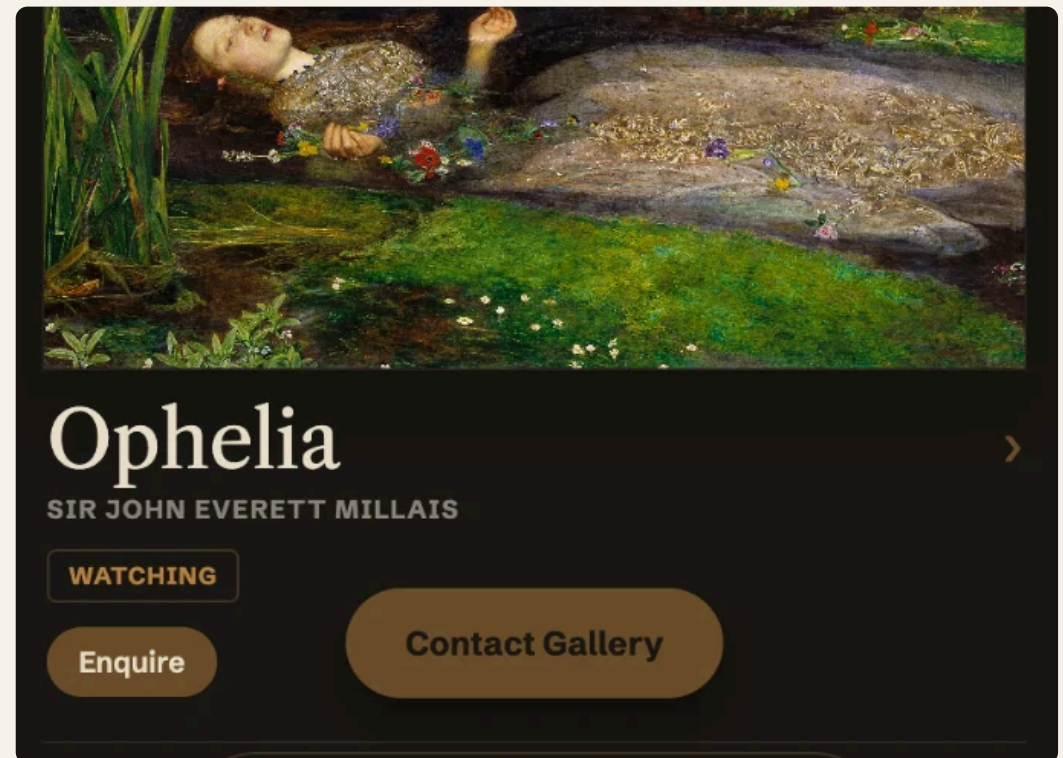
A hand-drawn promise: scan, read, listen, save.

Now the exhibit page, live



Served by `exhibit_preview_view` from real model data.

Now the collector dashboard



Watchlist, named collections and gallery inquiries — none of which existed at Update 1.

Two Django apps, one working product: *exhibits* and *users*.

exhibits · public

Scan → exhibit page

The public READ core: scan entry point, exhibit detail with TLDR, full text, audio & video — no login required.

exhibits · routing

Stable QR routes

Each exhibit auto-generates a printable QR identifier on first save; */exhibits/qr/1001/* resolves it forever.

users · auth

Full account flow

Register, login, logout and a complete password-reset flow — with the scanned exhibit carried through the whole journey.

users · dashboard

Collector dashboard

Watchlist saves, named collections (many-to-many), notes, and a profile with image upload & auto-resize.

users · leads

Inquiry → Prospect

A gallery inquiry mirrors into a Prospect record — the lead-capture moat from the pivot, now real code.

admin

Gallery CMS

Artists, artworks, shows and exhibits published through the Django admin — the operator needs no engineer.

Scan → save → *a new account*, end to end.

i. **Start as an anonymous visitor** — `/exhibits/`

Open the scan page and follow it to the newest exhibit. Point out: no login, this is the public READ core.

ii. **Tap save** → `/register/?interest_exhibit=...`

The URL carries the scanned exhibit's id — the register view stores it in the session so nothing is lost at the gate.

iii. **Submit the form** — **create the user live**

UserRegisterForm validates, `auth_login()` signs the new user in, and a success message confirms it.

iv. **Land on** `/dashboard/` — **context intact**

The scanned artwork is already waiting in the Scanned tab; save it to the watchlist with one click.

v. **Prove it in** `/admin/`

Show the new User row — and that Django stored a salted hash, never the password itself.

WHAT THIS DEMONSTRATES

Session-preserved context

`interest_exhibit` survives the whole auth flow — the lead-capture idea in code.

Django's auth framework

forms, validation, `login()`, `@login_required` — used, not reinvented.

Security by default

PBKDF2 password hashing and CSRF protection on every form.

The messages framework

every action confirms itself to the user.

One scan, four layers: *URL* → *view* → *model* → *template*.

URLCONF*Match the route*

```
# exhibits/urls.py
urlpatterns = [
    path('', views.scan_view,
         name='scan'),
    path(
        'qr/<int:qr_identifier>/',
        views.exhibit_preview_view,
        name='exhibit_preview_by_qr',
    ),
]
```

The path converter parses the QR number and hands it to the view.

VIEW*Handle the request*

```
# exhibits/views.py
def exhibit_preview_view(
    request, qr_identifier):
    exhibit = get_object_or_404(
        Exhibit,
        qr_identifier=qr_identifier)
    return render(request,
        'exhibits/base.html',
        {'featured_exhibit':
         exhibit})
```

Queries the ORM; a missing exhibit is a clean 404, not a crash.

MODEL*Own the data*

```
# exhibits/models.py
class Exhibit(models.Model):
    artwork = models.ForeignKey(
        Artwork, ...)
    qr_identifier = ... (
        unique=True, db_index=True)

    def save(self, ...):
        # qr = self.id + 1000
```

Business logic lives on the model — the QR number generates itself on first save.

TEMPLATE*Render the page*

```
<!-- exhibits/base.html -->
{{ featured_exhibit.tldr }}
{{ featured_exhibit.artist }}

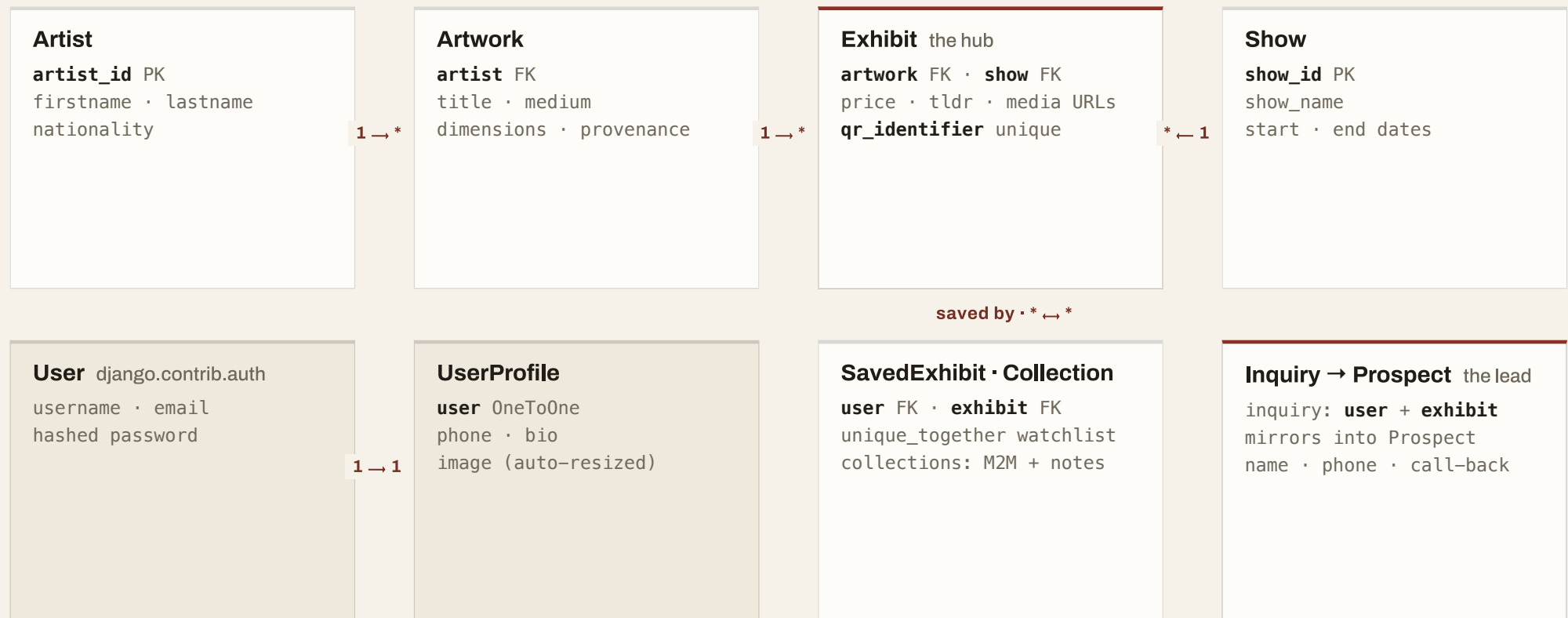
# .artist is a property
# that walks the relations:
# Exhibit → Artwork → Artist
```

Templates stay simple; model properties hide the joins.

THE POINT

Request `/exhibits/qr/1001/` and each layer does one job: URLconf routes, the view coordinates, the model owns data and logic, the template presents. That separation is exactly what makes the React + DRF split possible.

Eight models, two apps — art data *separated from* user activity.



The suite covers the two things that must not break: *models and routes.*

WHAT'S COVERED TODAY

Model defaults & labels

gallery default, currency choices, readable `__str__` for admin lists.

QR auto-generation

a new exhibit mints `qr_identifier = id + 1000`, exactly once.

Every public route

scan, preview-by-id and preview-by-QR return 200 with the right template.

Failure paths

unknown exhibit URLs return a clean 404 — never a silent wrong page.

Lead records

Prospect display text names the person and the artwork they want.

EXHIBITS/TESTS.PY

A model test & a route test

```
def test_qr_identifier_is_auto_generated(self):
    exhibit = Exhibit.objects.create(...)
    self.assertEqual(
        exhibit.qr_identifier, exhibit.id + 1000)

def test_qr_route_uses_base_template(self):
    response = self.client.get(
        reverse('exhibit_preview_by_qr',
                args=[exhibit.qr_identifier]))
    self.assertEqual(response.status_code, 200)
    self.assertTemplateUsed(
        response, 'exhibits/base.html')
```

The route test exercises the full chain from slide 06 — URLconf, view, model lookup and template — in one assertion.

Seven tests, one command, *green*.

- i.* **Run it** — `python manage.py test exhibits -v 2`
Verbose mode lists each test by name as it passes — narrate what each one proves.
- ii.* **Point at the test database line**
Django creates a throwaway database, runs everything inside it, and destroys it — the real data is never touched.
- iii.* **Open `exhibits/tests.py` on the QR test**
Walk one model test: create an exhibit in setUp-style, assert the generated identifier. Small, fast, specific.
- iv.* **Break it, run it, fix it (*if time allows*)**
Change the +1000 rule, watch the test fail with a precise message, revert. That is the safety net in action.

WHAT THIS DEMONSTRATES

TestCase isolation

each test runs in a transaction that rolls back — order can't matter.

reverse(), not hard-coded URLs

tests survive route changes because they resolve names, like templates do.

Model logic is testable logic

the QR rule lives on the model, so one unit test pins it down.

Ready for DRF

the same suite pattern extends to APIClient tests in the final stretch.

Every piece of feedback became *a change you can see in the repo.*

-
- | | | |
|-------------|--|---|
| <i>i.</i> | <p><i>“Sharpen the business case — who actually pays?”</i></p> <p>From the pitch reviews</p> | <p>Pivoted to commercial galleries — and wrote it into the code: <code>Prospect</code> and <code>GalleryInquiry</code> models, price & currency on every exhibit, and an inquiry flow that mirrors interest into a lead record the gallery can act on.</p> |
| <i>ii.</i> | <p><i>“Normalize the data model.”</i></p> <p>From Update 1</p> | <p>Split the flat exhibit record into <code>Artist</code>, <code>Artwork</code> and <code>Show</code> with proper foreign keys — and kept templates untouched by exposing compatibility properties on <code>Exhibit</code> that walk the relations.</p> |
| <i>iii.</i> | <p><i>“Prove it’s stable, not just that it runs.”</i></p> <p>From Update 1</p> | <p>Added the automated test suite you just saw — model logic, every public route, and failure paths — plus guarded 404s so a bad QR scan degrades cleanly instead of erroring.</p> |
-

Decouple the front end; the *models and tests carry over unchanged*.

Django templates today

→ **React PWA** *front end*

The scan → exhibit → dashboard journey rebuilt as components; installable, offline-cached, mobile-first.

Function-based views today

→ **DRF serializers + ViewSets**

The same querysets exposed as a clean JSON API — one layer serving the PWA now and the gallery dashboard later.

Session auth today

→ **JWT tokens**

dj-rest-auth for register / login / reset over the API — the same auth story, stateless for a decoupled client.

Wk 1 · API first

DRF scaffold; read-only exhibit endpoints; React app shell alongside the monolith.

Wk 2 · Public flow

React scan → exhibit page consuming the API; media served cleanly; < 2 s budget held.

Wk 3 · Auth + dashboard

JWT flow; watchlist, collections and inquiries rebuilt against the API.

Wk 4 · Harden & submit

APIClient test suite, documentation of the process, deployment, final submission.

WHY THIS IS LOW-RISK

Slide 06 is the argument: because data and logic already live in the **models**, swapping the presentation layer changes the edges, not the core. The test suite comes along to prove it.

That's where it stands — *over to you.*

A working Django app today; a decoupled React + DRF version by submission. I'd value your feedback most on the final-stretch plan and anything you'd like to see in the API design.

INTERACTIVE FEEDBACK — WHILE WE TALK



Scan, or visit

`http://<my-ip>:8000/exhibits/`

and try the flow yourself.

- i.* Does scan → story feel instant?
The < 2-second bar, on your phone, right now.
- ii.* Is the register-at-save gate fair?
You only meet it when you try to keep something.
- iii.* What should the React version do better?
Tell me before I build it, not after.